KoraDevs

Quand tu bloques, un développeur te rejoint. · Concours CADEV 2026 (Systalink)

Essayer en ligne : https://koradevs.onrender.com/ (le premier chargement peut prendre une minute)

Le problème

Un développeur bloque sur une erreur. Les forums répondent tard, l'IA tourne parfois en rond, et on se sent seul. Les développeurs prêts à aider ne savent pas qui a besoin d'eux, ni quand. Et une fois le bug réglé en message privé, le savoir disparaît.

L'idée

On colle son erreur. Un développeur disponible reçoit l'alerte en direct et rejoint une **salle de code partagée** : même code, chat, bouton pour lancer le test. Quand le test passe au vert :

- la solution devient une **fiche** que les suivants retrouvent (cause, correction, avant / après) ;
- l'aidant gagne une **preuve** dans son **Passeport**, vérifiable par n'importe qui avec un QR code et une empreinte SHA-256.

> **Pourquoi pas ChatGPT ou Copilot ?** KoraDevs ne les remplace pas. Il sert quand le bug dépend de ton contexte et qu'un humain doit voir ton code. Chaque aide devient en plus un savoir vérifié par un test, en français, que les suivants retrouvent.

Ce qu'on a construit

Fonction	Ce que ça fait
Demande d'aide	3 étapes : l'erreur et le code, un aperçu exact de ce qui part (secrets masqués), l'attente d'un voisin.
Radar	Les aidants de la même technologie sont alertés tout de suite, les technologies proches après 2 minutes, tous les aidants disponibles après 5 minutes.
Salle de code	Éditeur partagé en direct, chat, exécution dans le navigateur, bouton « Problème résolu ».
Fiches	Bibliothèque de solutions : recherche, « utile », commentaires, avant / après rejouable.
Passeport	Preuves d'aide confirmées par un test, empreinte SHA-256, QR code.
Espaces	Salons par technologie, suivi de développeurs, fil de leurs aides.
Projets	Présenter un projet et trouver des coéquipiers.

Parcours guidé (5 minutes)

> Ouvre **deux fenêtres** : une fenêtre normale (le **demandeur**, fenêtre A) et une fenêtre de navigation privée (l'**aidant**, fenêtre B). Tu vois ainsi les deux côtés sans avoir besoin de personne d'autre.

#	Où	Ce que tu fais	Ce que tu dois voir
1	B	Crée un compte (e-mail + mot de passe). Sur **Accueil**, choisis JavaScript, clique **Me rendre disponible**.	Ta porte est ouverte : tu peux être alerté.
2	A	Crée un autre compte. Clique **Demander de l'aide**, colle l'erreur et le code ci-dessous, puis ajoute la ligne `DB_PASSWORD=ghp_a1B2c3D4e5F6g7H8i9J0k1L2m3N4o5P6q7R8`.	À l'étape « Aperçu », la fausse clé est **masquée** avant l'envoi.
3	B	Une alerte apparaît : clique **Accepter**.	Tu entres dans la salle, le code est là.
4	B	Corrige la ligne 2, écris dans le chat.	La fenêtre A voit **le code et le chat bouger en direct**.
5	A	Clique **Exécuter**.	Le code tourne dans le navigateur, jamais sur le serveur. Un résultat vert lancé par le demandeur **confirme le test**.
6	A + B	Chacun clique **Problème résolu**, puis **Valider la fiche**.	La salle se ferme, le code est effacé du serveur.
7	A	Ouvre **Fiches**, puis la fiche créée.	Cause, correction, avant / après rejouable.
8	B	Ouvre **Moi**, puis ton **Passeport**. Clique **Vérifier l'empreinte**.	Le navigateur recalcule le SHA-256 : aucune confiance aveugle dans le serveur.

Code et erreur d'exemple (fichier `users.js`, JavaScript)

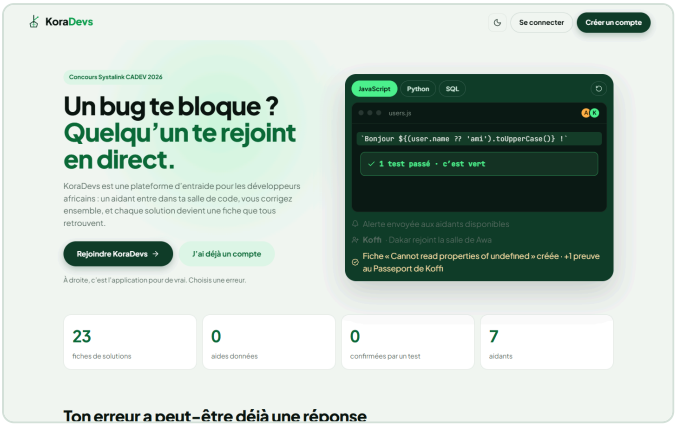
```
export const greet = (user) =>
  `Bonjour ${user.name.toUpperCase()} !`;

console.log(greet({}));
```

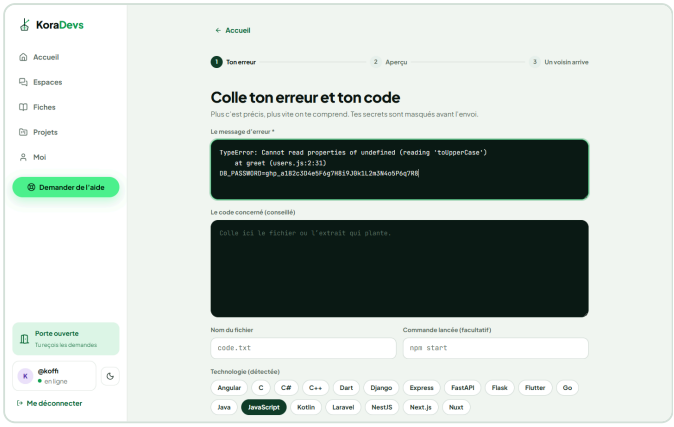
Erreur : `TypeError: Cannot read properties of undefined (reading 'toUpperCase')`
Correction à faire (ligne 2) : `` `Bonjour ${(user.name ?? 'ami').toUpperCase()} !`; ``

Pour aller plus vite : l'onglet **Fiches** contient 23 solutions déjà prêtes, et **Espaces** des salons par technologie.

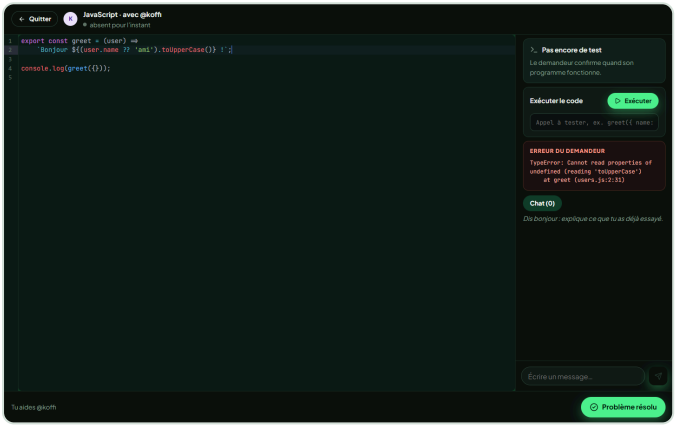
Ce que tu verras



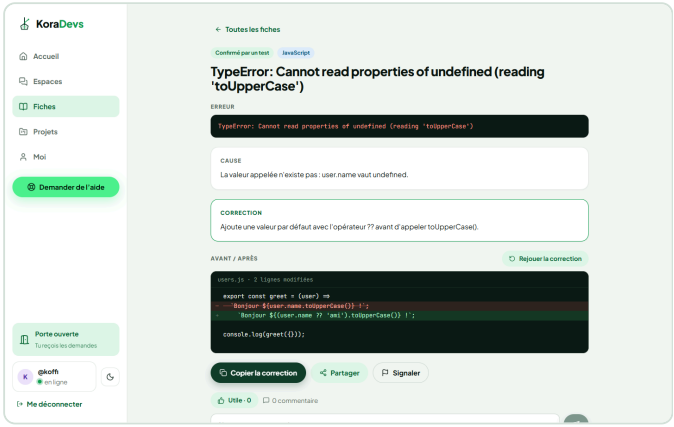
Accueil : la démonstration en direct



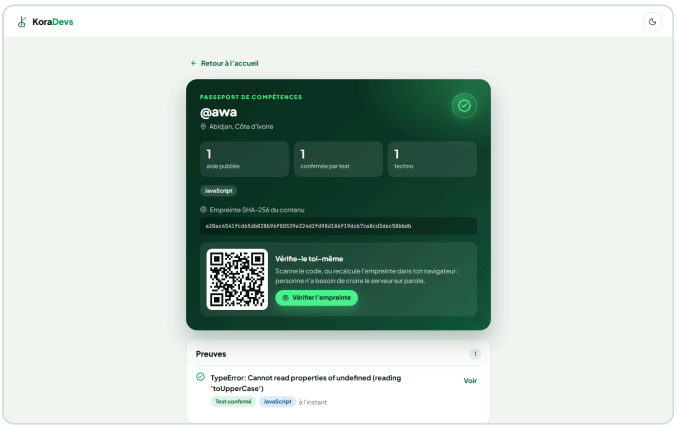
Demander de l'aide : l'erreur et le code



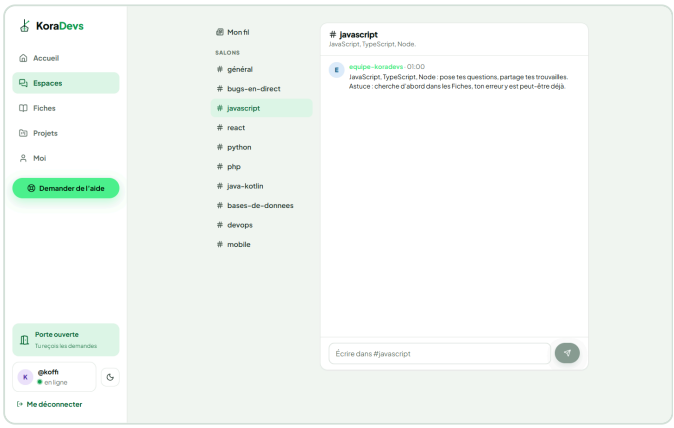
Salle de code partagée



Fiche avec avant / après



Passeport vérifiable (QR code, SHA-256)



Salons par technologie

Ce qui est réel, ce qui est préparé

Réel	Comptes (e-mail ou GitHub), alertes en direct (WebSocket), salle partagée (Yjs), chat, masquage des secrets, exécution du code dans le navigateur, fiches, avant / après, Passeport et empreinte SHA-256, salons, suivi, commentaires.
Préparé par l'équipe	23 fiches de départ, pour que la bibliothèque ne soit pas vide, et un message d'accueil dans chaque salon, signé `equipe-koradevs`.
Jamais inventé	Aucun faux compte, aucune fausse preuve, aucun aidant automatique. Les aidants et les preuves du Passeport viennent de vraies personnes et de vrais tests.
Limites actuelles	Pas d'IA : le concept initial prévoyait une « piste de l'IA », elle n'est pas dans le produit. Pas de vérification d'e-mail ni de « mot de passe oublié ». Pas de compte administrateur ni de modération. Seul le JavaScript s'exécute dans la salle. L'offre gratuite de Render s'endort après un moment d'inactivité.

Sécurité et confidentialité

- Les secrets (mots de passe, clés, jetons) sont masqués avant l'envoi, puis une seconde fois par le serveur.
- Seuls le demandeur et l'aidant qui a accepté voient le code. Il est effacé à la fin de la session, 24 heures au maximum.
- Le serveur n'exécute jamais de code : l'exécution se fait dans un espace isolé du navigateur (5 secondes au plus).
- Seul un résultat vert lancé par le demandeur confirme le test : l'aidant ne peut pas se confirmer lui-même.
- Les mots de passe des comptes ne sont jamais gardés, seulement leur empreinte (scrypt).

Technologies

Serveur **NestJS** (Node, TypeScript) avec WebSocket · base **PostgreSQL** · site **React + Vite** · éditeur **CodeMirror 6** synchronisé avec **Yjs**. Toutes les dépendances sont sous licence permissive (aucune GPL, AGPL ou LGPL), vérifié automatiquement à chaque push. Le code source est sur GitHub (dépôt *Koradevs*, documentation dans le README).